VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

Abdul Khadar D Jamadar (1BM24CS005)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by **Abdul khadar D Jamadar (1BM24CS005)** , who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 .The Lab report has been approved as it satisfies the academic requirements in respect of OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program 1 : FCFS & SJF

Question : Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.
→FCFS
→ SJF (pre-emptive & Non-preemptive)

A.Code :

FCFS :

```
//program to simulate FCFS scheduling
#include <stdio.h>


void sort(int* at,int* bt,int* p,int n){
    for(int i=0;i<n;i++){
        for(int j=0;j<n-i-1;j++){
            if(at[j] > at[j+1]){
                int temp=at[j]; at[j] = at[j+1]; at[j+1] = temp;
                temp = bt[j]; bt[j] = bt[j+1]; bt[j+1] = temp;
                temp = p[j]; p[j] = p[j+1] ; p[j+1] = temp;
            }
        }
    }
}

int main(){
    int n;
    printf("Enter the number of process: ");
    scanf("%d",&n);
```

```
    int at[n],bt[n],ct[n],tat[n],wt[n],p[n];

    for(int i=0;i<n;i++){
        printf("Enter the AT and BT of P%d : ",i+1);
        p[i] = i+1;
        scanf("%d %d",&at[i],&bt[i]);
    }

    sort(at,bt,p,n);

    ct[0] = at[0] + bt[0];

    for(int i=1;i<n;i++){
        if(ct[i-1] > at[i])
            ct[i] = ct[i-1] + bt[i];
        else
            ct[i] = at[i] + bt[i];
    }
    int sum_tat = 0,sum_wt = 0;
    printf("\n\nProcess \tAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++){
        tat[i] = ct[i]-at[i];
        wt[i] = tat[i] - bt[i];
        sum_tat += tat[i];
        sum_wt += wt[i];
        printf("p%d
\t\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\n\n");

    float avg_tat = (float)sum_tat/n,avg_wt = (float)sum_wt/n;
```

```
    printf("Average TAT: %.2f\nAverage WT: %.2f",avg_tat,avg_wt);
    printf("\n\nPROGRAM DEVELOPED BY ABDUL KHADAR JAMADAR");
    return 0;
}
```

SJF :

```
// program to simulate SJF algorithm
#include <stdio.h>

void sort(int* at,int* bt,int* p,int n){
    for(int i=0;i<n;i++){
        for(int j=0;j<n-i-1;j++){
            if(at[j] > at[j+1]){
                int temp=at[j]; at[j] = at[j+1]; at[j+1] = temp;
                temp = bt[j]; bt[j] = bt[j+1]; bt[j+1] = temp;
                temp = p[j]; p[j] = p[j+1] ; p[j+1] = temp;
            }
        }
    }
}

int main(){
    int n;
    printf("Enter the number of process: ");
    scanf("%d",&n);

    int at[n],bt[n],ct[n],tat[n],wt[n],p[n],finished[n];

    for(int i=0;i<n;i++){
        printf("Enter the AT and BT of P%d : ",i+1);
        p[i] = i+1;
        scanf("%d %d",&at[i],&bt[i]);
        finished[i] = 0;
    }
```

```
sort(at,bt,p,n);

int completed=0,current_time = 0;

while(completed < n){
    int idx=-1;
    int min_bt=9999;

    for(int i=0;i<n;i++){
        if(at[i]<=current_time && finished[i] == 0){
                if(bt[i] < min_bt){
                    min_bt = bt[i];
                    idx = i;
                }
        }
    }

    if(idx != -1) {
        ct[idx] = current_time + bt[idx];
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        current_time = ct[idx];
        finished[idx] = 1;
        completed++;
    }
    else
        current_time++;
}
```

```
    int sum_tat = 0,sum_wt = 0;

    printf("\n\nProcess \tAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0;i<n;i++){
        sum_tat += tat[i];
        sum_wt += wt[i];
        printf("p%d
\t\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\n\n");

    float avg_tat = (float)sum_tat/n,avg_wt = (float)sum_wt/n;
    printf("Average TAT: %.2f\nAverage WT: %.2f",avg_tat,avg_wt);
    printf("\n\nPROGRAM DEVELOPED BY ABDUL KHADAR JAMADAR");
}
```

SRTF :

```
#include <stdio.h>
#include <limits.h>


int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], p[n];
```

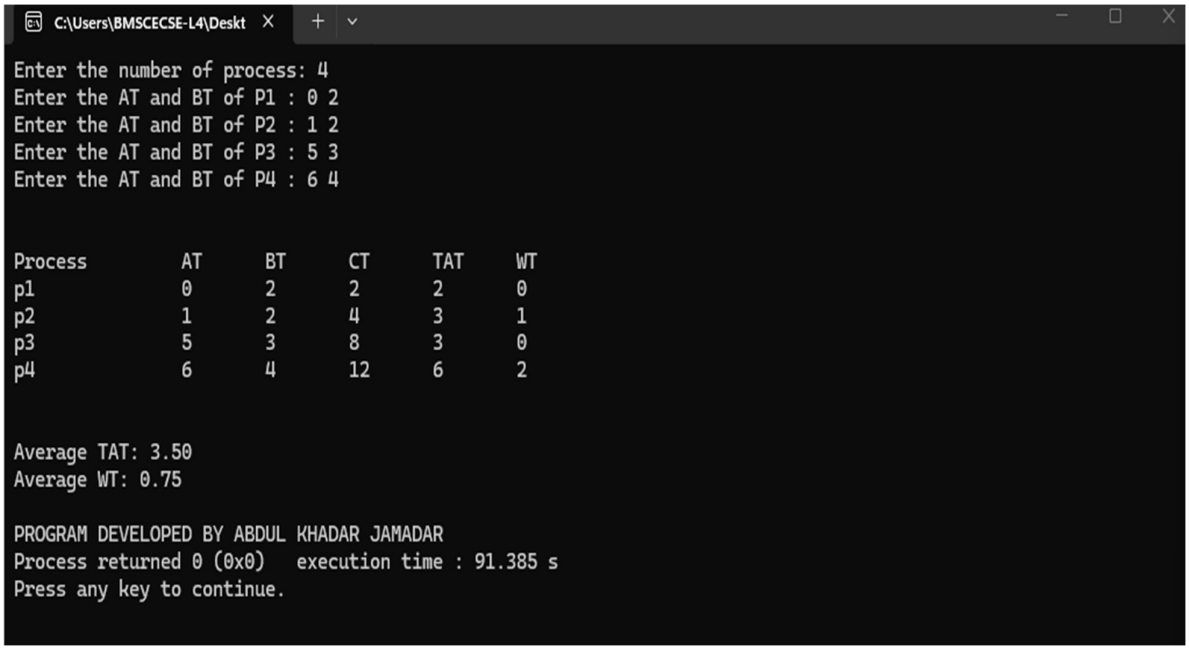
```
for (int i = 0; i < n; i++) {
    printf("Enter AT and BT for P%d: ", i + 1);
    scanf("%d %d", &at[i], &bt[i]);
    p[i] = i + 1;
}



for (int i = 0; i < n; i++) {
    rt[i] = bt[i];
}

int current_time = 0, completed = 0;

while (completed < n) {
    int idx = -1;
    int min_rt = INT_MAX;


    for (int i = 0; i < n; i++) {
        if (at[i] <= current_time && rt[i] > 0) {
            if (rt[i] < min_rt) {
                min_rt = rt[i];
                idx = i;
            }

            else if (rt[i] == min_rt) {
                if (at[i] < at[idx]) {
                    idx = i;
                }
            }
```

```
            }
        }

        if (idx == -1) {
            current_time++;
        } else {
            rt[idx]--;
            current_time++;

            if (rt[idx] == 0) {
                completed++;
                ct[idx] = current_time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];
            }
        }
    }


    float total_tat = 0, total_wt = 0;

    printf("\n\nProcess \tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        total_tat += tat[i];
        total_wt += wt[i];
        printf("P%d \t\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i],
bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\n----------------------------------------");
    printf("\nTotal Turnaround Time: %.2f", total_tat);
    printf("\nTotal Waiting Time:    %.2f", total_wt);
```

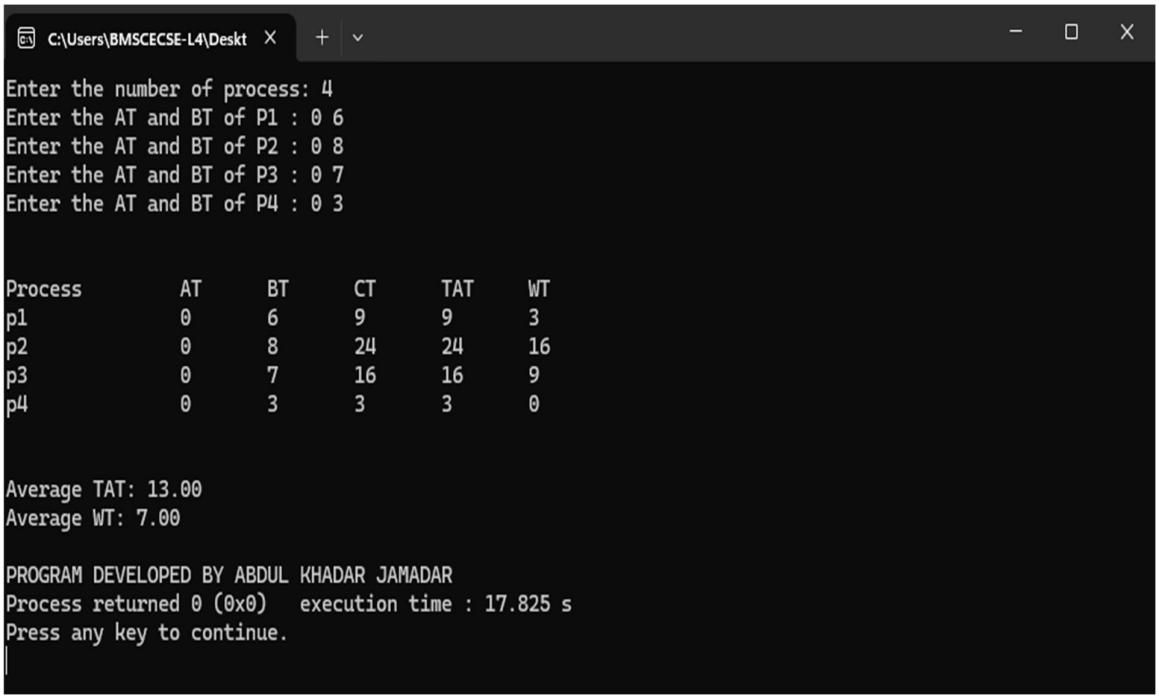
```
    printf("\nAverage Turnaround Time: %.2f", total_tat / n);
    printf("\nAverage Waiting Time:    %.2f", total_wt / n);
    printf("\n----------------------------------------");
    printf("\n\nPROGRAM DEVELOPED BY ABDUL KHADAR JAMADAR\n");

    return 0;
}
```

B.Output :

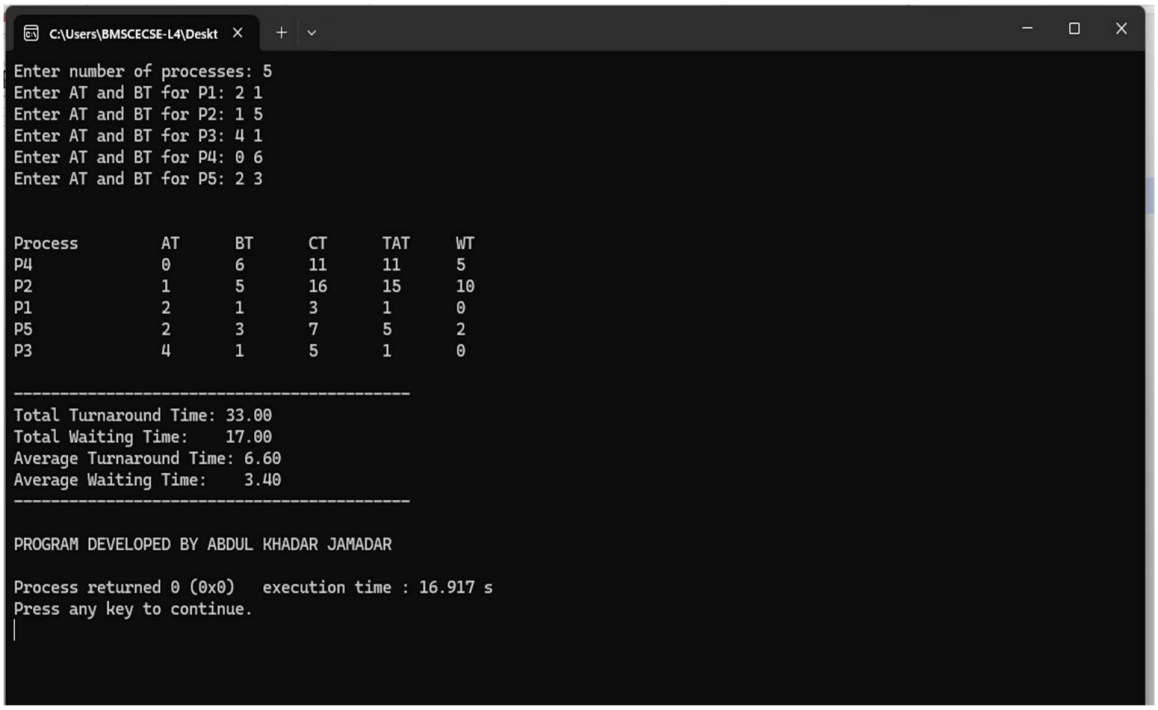
FCFS



SJF:



SRTF (SJF preemptive)



PROGRAM 2 : PRIORITY (PREEMPTIVE&NON-PREEMPTIVE) ROUND ROBIN (diff time quantum)

Question : Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)
→Round Robin (Experiment with different quantum sizes for RR algorithm)

A.Priority preemptive :

```
// Program to simulate the preemptive scheduling algorithms
// assuming LOWER THE NUMBER -> HIGHER THE PRIORITY
#include <stdio.h>

int main(){
    int n;
    printf("Enter the number of process : ");
    scanf("%d",&n);

    int arr[n],at[n],bt[n],ct[n],wt[n],rt[n],pr[n],p[n],tat[n];

    for(int i=0;i<n;i++){
        printf("Enter the AT,BT and priority of P%d : ",i+1);
        p[i] = i+1;
        scanf("%d %d %d",&at[i],&bt[i],&pr[i]);
        rt[i] = bt[i];
    }
```

```
    int completed = 0,current_time = 0;

    while(completed < n){
        int idx = -1;
        int highest = 9999;

        for(int i=0;i<n;i++){
            if(at[i] <= current_time && rt[i] > 0  && pr[i] <
highest){
                highest = pr[i];
                idx = i;
            }
        }

        if(idx != -1){
            rt[idx]--;
            current_time++;

            if(rt[idx] == 0){
                ct[idx] = current_time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];
                completed++;
            }
        }
        else
            current_time++;
    }

    float total_tat = 0, total_wt = 0;

    printf("\n\nProcess \tAT\tBT\tPRIORITY\tCT\tTAT\tWT\n");
```

```
    for (int i = 0; i < n; i++) {
        total_tat += tat[i];
        total_wt += wt[i];
        printf("P%d \t\t%d\t%d\t%d\t\t%d\t%d\t%d\n",
p[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage TAT: %.2f", total_tat / n);
    printf("\nAverage WT:    %.2f", total_wt / n);

    printf("\n\nPROGRAM DEVELOPED BY ABDUL KHADAR JAMADAR\n");
    return 0;
}
```

B.Priority Non-preemptive :

```
// Program to simulate the non preemptive scheduling algorithms
// assuming LOWER THE NUMBER -> HIGHER THE PRIORITY
#include <stdio.h>



int main(){
    int n;
    printf("Enter the number of process : ");
    scanf("%d",&n);

    int
arr[n],at[n],bt[n],ct[n],wt[n],pr[n],p[n],tat[n],finished[n];

    for(int i=0;i<n;i++){
```

```
        printf("Enter the AT,BT and priority of P%d : ",i+1);
        p[i] = i+1;
        scanf("%d %d %d",&at[i],&bt[i],&pr[i]);
        finished[i] = 0;
    }

    int completed = 0,current_time = 0;

    while(completed < n){
        int idx = -1;
        int highest = 9999;

        for(int i=0;i<n;i++){
            if(at[i] <= current_time && finished[i] == 0 && pr[i]
< highest){
                highest = pr[i];
                idx = i;
            }
        }

        if(idx != -1){
                ct[idx] = current_time+bt[idx];
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];
                current_time = ct[idx];
                finished[idx] = 1; // marking the process as
finished
                completed++;
            }
        else
            completed++;
    }
```

```
    float total_tat = 0, total_wt = 0;

    printf("\n\nProcess \tAT\tBT\tPRIORITY\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        total_tat += tat[i];
        total_wt += wt[i];
        printf("P%d \t\t%d\t%d\t%d\t\t%d\t%d\t%d\n",
p[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage TAT: %.2f", total_tat / n);
    printf("\nAverage WT:    %.2f", total_wt / n);

    printf("\n\nPROGRAM DEVELOPED BY ABDUL KHADAR JAMADAR\n");
    return 0;
}
```

C.Round Robin :

```
#include <stdio.h>

int main() {
    int n, tq;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], p[n];
    int visited[n];
```

```
for (int i = 0; i < n; i++) {
    p[i] = i + 1;
    printf("Enter AT and BT of P%d: ", p[i]);
    scanf("%d %d", &at[i], &bt[i]);
    rt[i] = bt[i];
    visited[i] = 0;
}

int queue[100]; // Simple queue array
int front = 0, rear = 0;
int completed = 0, current_time = 0;


while(completed < n) {
    for(int i = 0; i < n; i++) {
        if(at[i] <= current_time && visited[i] == 0) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    if (front == rear) {
        current_time++;
        continue;
    }

    int i = queue[front++];

    if (rt[i] > tq) {
        current_time += tq;
        rt[i] -= tq;
    } else {
```

```
            current_time += rt[i];
            rt[i] = 0;
            ct[i] = current_time;
            tat[i] = ct[i] - at[i];
            wt[i] = tat[i] - bt[i];
            completed++;
        }


        for(int j = 0; j < n; j++) {
            if(at[j] <= current_time && visited[j] == 0) {
                queue[rear++] = j;
                visited[j] = 1;
            }
        }


        if (rt[i] > 0) {
            queue[rear++] = i;
        }
    }
    float sum_tat = 0, sum_wt = 0;
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        sum_tat += tat[i];
        sum_wt += wt[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i],
ct[i], tat[i], wt[i]);
    }

    printf("\nAverage TAT: %.2f", sum_tat / n);
    printf("\nAverage WT: %.2f\n", sum_wt / n);
```

```
    return 0;
}
```

D.Output :

PRIORITY PREEMPTIVE :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\preem.exe
Enter the number of process : 3
Enter the AT,BT and priority of P1 : 0 5 3
Enter the AT,BT and priority of P2 : 1 2 1
Enter the AT,BT and priority of P3 : 2 4 2


Process         AT      BT      PRIORITY        CT      TAT     WT
P1              0       5       3               11      11      6
P2              1       2       1               3       2       0
P3              2       4       2               7       5       1

Average TAT: 6.00
Average WT:    2.33

PROGRAM DEVELOPED BY ABDUL KHADAR JAMADAR
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

PRIORITY NON-PREEMPTIVE :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\NON_PREEMP_PRIORITY.exe
Enter the number of process : 5
Enter the AT,BT and priority of P1 : 0 3 5
Enter the AT,BT and priority of P2 : 2 2 3
Enter the AT,BT and priority of P3 : 3 5 2
Enter the AT,BT and priority of P4 : 4 4 4
Enter the AT,BT and priority of P5 : 6 1 1


Process         AT      BT      PRIORITY        CT      TAT     WT
P1              0       3       5               3       3       0
P2              2       2       3               11      9       7
P3              3       5       2               8       5       0
P4              4       4       4               15      11      7
P5              6       1       1               9       3       2

Average TAT: 6.20
Average WT:    3.20

PROGRAM DEVELOPED BY ABDUL KHADAR JAMADAR
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

ROUND ROBIN :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\round.exe
Enter the number of processes: 5
Enter Time Quantum: 2
Enter AT and BT of P1: 0 5
Enter AT and BT of P2: 1 3
Enter AT and BT of P3: 2 1
Enter AT and BT of P4: 3 2
Enter AT and BT of P5: 4 3

PID     AT      BT      CT      TAT     WT
P1      0       5       13      13      8
P2      1       3       12      11      8
P3      2       1       5       3       2
P4      3       2       9       6       4
P5      4       3       14      10      7

Average TAT: 8.60
Average WT: 5.80
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\round.exe
Enter the number of processes: 5
Enter Time Quantum: 3
Enter AT and BT of P1: 0 5
Enter AT and BT of P2: 1 3
Enter AT and BT of P3: 2 1
Enter AT and BT of P4: 3 2
Enter AT and BT of P5: 4 3

PID     AT      BT      CT      TAT     WT
P1      0       5       14      14      9
P2      1       3       6       5       2
P3      2       1       7       5       4
P4      3       2       11      8       6
P5      4       3       12      8       5

Average TAT: 8.00
Average WT: 5.20
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

PROGRAM 3 : MULTILEVEL QUEUE

Question : Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

=>

A.Code :

```
#include <stdio.h>

int main() {
    int n, tq, time = 0, done = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    int p[n][7];
    for (int i = 0; i < n; i++) {
        p[i][0] = i + 1;
        printf("P%d [AT BT Type]: ", p[i][0]);
        scanf("%d %d %d", &p[i][1], &p[i][2], &p[i][6]);
        p[i][3] = p[i][2];
        p[i][4] = 0;
    }
```

```
    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tType\n");

    while (done < n) {
        int executed = 0;
        for (int i = 0; i < n; i++) {
            if (p[i][1] <= time && !p[i][4]) {
                executed = 1;
                int slice = (p[i][3] > tq) ? tq : p[i][3];
                time += slice;
                p[i][3] -= slice;

                if (p[i][3] == 0) {
                    p[i][5] = time;
                    p[i][4] = 1;
                    done++;
                    int tat = p[i][5] - p[i][1];
                    int wt = tat - p[i][2];
                    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%s\n",
                        p[i][0], p[i][1], p[i][2], p[i][5], tat,
wt,
                        (p[i][6] == 1) ? "System" : "User");
                }
            }
        }
        if (!executed) time++;
    }

    float total_tat = 0, total_wt = 0;
    for (int i = 0; i < n; i++) {
        total_tat += (p[i][5] - p[i][1]);
        total_wt += (p[i][5] - p[i][1] - p[i][2]);
    }
```

```
    printf("\nAverage TAT: %.2f", total_tat / n);
    printf("\nAverage WT : %.2f\n", total_wt / n);

    return 0;
}
```

B.Output :

```
PS D:\BMSCE\OS> .\multi.exe
Enter the number of processes: 4
Enter Time Quantum: 2

Enter AT, BT & Type (1:System, 2:User) for P1: 0 4 1
Enter AT, BT & Type (1:System, 2:User) for P2: 0 3 1
Enter AT, BT & Type (1:System, 2:User) for P3: 0 8 2
Enter AT, BT & Type (1:System, 2:User) for P4: 10 5 1

Process AT      BT      CT      TAT     WT      Type
P2      0       3       7       7       4       System
P1      0       4       8       8       4       System
P4      10      5       20      10      5       System
P3      0       8       21      21      13      User

Average TAT: 11.50
Average WT : 6.50

PS D:\BMSCE\OS> 1BM24CS005
```

PROGRAM 4 : REAL TIME CPU SCHEDULING ALGORITHMS (RATE MONOTONIC,EARLIEST DEADLINE,PROPORTIONAL)

Question : Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

=>

A.Rate Monotonic :

```
#include <stdio.h>
#include <math.h>

int find_lcm(int a, int b) {
    int max = (a > b) ? a : b;
    while (1) {
        if (max % a == 0 && max % b == 0) return max;
        max++;
    }
}

int main() {
    int n, i, j, current_time = 0, hyperperiod = 1;
    float utilization = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);
```

```
    int burst[n], period[n], remaining_burst[n];

    printf("Enter the CPU burst times:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &burst[i]);
        remaining_burst[i] = burst[i];
    }

    printf("Enter the time periods:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &period[i]);
        if (i == 0) hyperperiod = period[i];
        else hyperperiod = find_lcm(hyperperiod, period[i]);
        utilization += (float)burst[i] / period[i];
    }

    printf("LCM = %d\n", hyperperiod);
    printf("\nRate Monotone Scheduling:\n");
    printf("PID\tBurst\tPeriod\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
    }

    printf("\n%f <= %f => true\n\n", utilization, n * (pow(2,
1.0/n) - 1));
    printf("Scheduling occurs for %d ms\n", hyperperiod);

    int last_pid = -1;
    while (current_time < hyperperiod) {
        int selected = -1;
        int min_period = 10000;
```

```
        for (i = 0; i < n; i++) {
            if (current_time % period[i] == 0) {
                remaining_burst[i] = burst[i];
            }
        }

        for (i = 0; i < n; i++) {
            if (remaining_burst[i] > 0) {
                if (period[i] < min_period) {
                    min_period = period[i];
                    selected = i;
                }
            }
        }

        if (selected != last_pid) {
            if (selected != -1)
                printf("%dms onwards: Process %d running\n",
current_time, selected + 1);
            else
                printf("%dms onwards: CPU is idle\n",
current_time);
            last_pid = selected;
        }

        if (selected != -1) remaining_burst[selected]--;
        current_time++;
    }
    printf("\n\nProgram Developed by : 1bm24cs005");
    return 0;
}
```

B.Earliest Deadline first :

```
#include <stdio.h>

int main() {
    int n, i, j, time = 0, completed = 0, total_time;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int burst[n], period[n], deadline[n], remaining[n],
next_deadline[n];

    for (i = 0; i < n; i++) {
        printf("Enter Burst, Period, and Deadline for P%d: ", i +
1);
        scanf("%d %d %d", &burst[i], &period[i], &deadline[i]);
        remaining[i] = burst[i];
        next_deadline[i] = deadline[i];
    }

    printf("Enter total simulation time: ");
    scanf("%d", &total_time);

    printf("\nScheduling occurs for %d ms\n", total_time);

    int last_pid = -1;
    while (time < total_time) {
        int selected = -1;
        int min_deadline = 10000;

        for (i = 0; i < n; i++) {
            if (time > 0 && time % period[i] == 0) {
                remaining[i] = burst[i];
```

```
                next_deadline[i] = time + deadline[i];
            }
        }

        for (i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                if (next_deadline[i] < min_deadline) {
                    min_deadline = next_deadline[i];
                    selected = i;
                }
            }
        }

        if (selected != last_pid) {
            if (selected != -1)
                printf("%dms: Task %d is running.\n", time,
selected + 1);
            else
                printf("%dms: CPU is idle.\n", time);
            last_pid = selected;
        }

        if (selected != -1) remaining[selected]--;
        time++;
    }
    return 0;
}
```

C.Proportional Scheduling :

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct {
    int id;
    int weight;
    int stride;
    int pass;
} Process;

void runStrideScheduling(Process processes[], int n, int
totalSteps) {
    // Calculate stride: 10000 / weight
    // Smaller stride = larger share = runs more often!
    for (int i = 0; i < n; i++) {
        processes[i].stride = 10000 / processes[i].weight;
        processes[i].pass = 0;
    }

    for (int i = 0; i < totalSteps; i++) {
        // Find process with the smallest pass value
        int best = 0;
        for (int j = 1; j < n; j++) {
            if (processes[j].pass < processes[best].pass) {
                best = j;
            }
        }

        printf("Running Process P%d (Pass: %d)\n",
               processes[best].id, processes[best].pass);

        // Advance the pass
        processes[best].pass += processes[best].stride;
    }
}
```

```
int main() {
    // Define processes: ID, Weight
    Process processes[] = {{1, 10}, {2, 20}, {3, 5}};
    int n = sizeof(processes) / sizeof(processes[0]);

    runStrideScheduling(processes, n, 10);

    return 0;
}
```

D.Output :

RMS :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> ./rms.exe
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM = 100

Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true

Scheduling occurs for 100 ms
0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle


Program Developed by : 1bm24cs005
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

Earliest Deadline first :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\edf.exe
Enter number of processes: 3
Enter Burst, Period, and Deadline for P1: 2 4 5
Enter Burst, Period, and Deadline for P2: 3 7 10
Enter Burst, Period, and Deadline for P3: 2 8 10
Enter total simulation time: 20

Scheduling occurs for 20 ms
0ms: Task 1 is running.
2ms: Task 2 is running.
4ms: Task 1 is running.
6ms: Task 2 is running.
7ms: Task 3 is running.
8ms: Task 1 is running.
10ms: Task 2 is running.
12ms: Task 1 is running.
14ms: Task 3 is running.
16ms: Task 1 is running.
18ms: Task 2 is running.
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

Proportional Scheduling :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\stride_scheduler.exe
Calculating Strides based on hardcoded weights (P1:10, P2:20, P3:5):
P1 Stride = 1000
P2 Stride = 500
P3 Stride = 2000

Steps to complete (hardcoded totalSteps: 10):
Running Process P1 (Pass: 0)
Running Process P2 (Pass: 0)
Running Process P3 (Pass: 0)
Running Process P2 (Pass: 500)
Running Process P1 (Pass: 1000)
Running Process P2 (Pass: 1000)
Running Process P2 (Pass: 1500)
Running Process P1 (Pass: 2000)
Running Process P2 (Pass: 2000)
Running Process P3 (Pass: 2000)
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

PROGRAM 5 : PRODUCER-CONSUMER PROBLEM SEMAPHORE EDITION

Question : Write a C program to simulate producer-consumer problem using semaphores

=>

A.Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *param) {
    int item;
    while (1) {
        item = rand() % 100;
        sem_wait(&empty);
```

```
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer produced: %d at %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
}

void *consumer(void *param) {
    int item;
    while (1) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed: %d at %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
}

int main() {
    pthread_t prod_tid, cons_tid;

    sem_init(&empty, 0, BUFFER_SIZE);
```

```
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod_tid, NULL, producer, NULL);
    pthread_create(&cons_tid, NULL, consumer, NULL);

    pthread_join(prod_tid, NULL);
    pthread_join(cons_tid, NULL);

    return 0;
}
```

B.Output :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\stride_scheduler.exe
Calculating Strides based on hardcoded weights (P1:10, P2:20, P3:5):
P1 Stride = 1000
P2 Stride = 500
P3 Stride = 2000

Steps to complete (hardcoded totalSteps: 10):
Running Process P1 (Pass: 0)
Running Process P2 (Pass: 0)
Running Process P3 (Pass: 0)
Running Process P2 (Pass: 500)
Running Process P1 (Pass: 1000)
Running Process P2 (Pass: 1000)
Running Process P2 (Pass: 1500)
Running Process P1 (Pass: 2000)
Running Process P2 (Pass: 2000)
Running Process P3 (Pass: 2000)
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005>
```

PROGRAM 6 : DINING PHILOSOPHER'S PROBLEM

Question : Write a C program to simulate the concept of Dining Philosophers problem.

=>

A.Code :

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t fork_mutex[N];

void *philosopher(void *num) {
    int i = *(int *)num;

    while (1) {
        printf("Philosopher %d is thinking.\n", i);
        sleep(1);

        printf("Philosopher %d is hungry.\n", i);

        sem_wait(&fork_mutex[i]);
        sem_wait(&fork_mutex[(i + 1) % N]);

        printf("Philosopher %d is eating.\n", i);
```

```
        sleep(2);

        sem_post(&fork_mutex[i]);
        sem_post(&fork_mutex[(i + 1) % N]);

        printf("Philosopher %d finished eating.\n", i);
    }
}

int main() {
    pthread_t thread[N];
    int phil_id[N];

    for (int i = 0; i < N; i++)
        sem_init(&fork_mutex[i], 0, 1);

    for (int i = 0; i < N; i++) {
        phil_id[i] = i;
        pthread_create(&thread[i], NULL, philosopher,
&phil_id[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(thread[i], NULL);

    return 0;
}
```

B.Output :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\producer_consumer.exe
Producer produced: 42 at 0
Consumer consumed: 42 at 0
Producer produced: 87 at 1
Producer produced: 15 at 2
Consumer consumed: 87 at 1
Producer produced: 33 at 3
Producer produced: 56 at 4
[Buffer Full]
Consumer consumed: 15 at 2
Consumer consumed: 33 at 3
Producer produced: 91 at 0
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\producer_consumer.exe
```

PROGRAM 7 : BANKERS ALGORITHM

Question : Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

=>

A.Code :

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], avail[m];
    int f[n], ans[n], ind = 0;

    printf("Enter Allocation Matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for (j = 0; j < m; j++) scanf("%d", &avail[j]);
```

```
    int need[n][m];
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    for (k = 0; k < n; k++) f[k] = 0;

    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            if (f[i] == 0) {
                int flag = 0;
                for (j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]) {
                        flag = 1; break;
                    }
                }
                if (flag == 0) {
                    ans[ind++] = i;
                    for (j = 0; j < m; j++) avail[j] +=
alloc[i][j];
                    f[i] = 1;
                }
            }
        }
    }
    printf("Following is the SAFE Sequence:\n");
    for (i = 0; i < n - 1; i++) printf(" P%d ->", ans[i]);
    printf(" P%d\n", ans[n - 1]);

    return 0;
}
```

B.Output :

```
PS C:\Users\BMSCECSE-L4\Desktop\1BM24CS005> .\bankers.exe
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0 2 0 0 3 0 2 2 1 1
2 1 1 0 0 2 0 0 2

Enter Max Matrix:
7 5 3 3 2 2 9 0 2 0 2  2 2 2 4 3 3
Enter Available Resources: 3 3 2
Current Need Matrix:
7 4 3 1 2 2  6 0 0    0 1 1 4 3 1
[cite: Calculation in progress... Checking process P0... Need > Avail. [Waiting]]
[cite: Checking process P1... Need <= Avail. P1 runs. Avail += Alloc(P1). New Avail: 5 3 2]
[cite: Checking process P2... Need <= Avail. P2 runs. Avail += Alloc(P2). New Avail: 8 3 4]
[cite: Checking process P3... Need <= Avail. P3 runs. Avail += Alloc(P3). New Avail: 10 4 5]
[cite: Checking process P4... Need <= Avail. P4 runs. Avail += Alloc(P4). New Avail: 10 4 7]
[cite: Re-checking P0... Need <= Avail. P0 runs. Avail += Alloc(P0). Final Avail: 10 5 7]
Following is the SAFE Sequence:
P1 -> P2 -> P3 -> P4 -> P0
```